A Minimalist Putback-Based Bidirectional Programming Language, Formally Verified

Hsiang-Shang Ko[2], Tao Zan[1], and Zhenjiang Hu[1, 2]

[1] SOKENDAI (The Graduate University for Advanced Studies), Japan
{taozan,hu}@nii.ac.jp
[2] National Institute of Informatics, Japan
hsiang-shang@nii.ac.jp

1 Introduction

Theory construction in terms of Agda programming

2 Decidable partiality

The bidirectional transformations discussed in this paper are "decidably partial". By "decidable partiality" we mean that the transformations are actually total functions which wrap their results in the standard Maybe datatype:

```
data Maybe (A : Set) : Set where
  just    : A → Maybe A
  nothing : Maybe A
```

The result of a piece of successful computation is wrapped in the just constructor, while failed computation is represented by nothing. The behaviour of decidably partial functions is different from the usual partial functions as formalised in terms of complete partial orders (see, e.g., Gibbons [4]): In Section 4.3 we will need to define programs that produce some result or fail respectively when a sub-program fails or produces some result, but this is not possible in the setting of complete partial orders (since such functions violate monotonicity, a necessary condition for computability).

We can choose to write our decidably partial bidirectional transformations directly as Maybe-programs, but when proving their well-behavedness, we will need to analyse or establish how a piece of decidably partial computation succeeds or fails, and the task would be much easier if the computation steps leading to success or failure are reified as data. (An example of such proofs is given in the definition of lens composition _↔_ in Section 3.) The reification starts from the following datatype Par, which is a deep embedding of the combinators that we will use for writing decidably partial programs:

```
data Par : Set → Set₁ where
  return : {A : Set} → A → Par A
```

```
runPar : { A : Set } → Par A → Maybe A
runPar (return x) = just x
runPar (mx >>= f) with runPar mx
runPar (mx >>= f) | just x    = runPar (f x)
runPar (mx >>= f) | nothing = nothing
runPar fail = nothing
runPar (catch mx f my) with runPar mx
runPar (catch mx f my) | just x    = runPar (f x)
runPar (catch mx f my) | nothing = runPar my
runPar (assert true then mx) = runPar mx
runPar (assert false then mx) = nothing
```

Fig. 1. Definition of the interpreter of Par to Maybe

```
_>>=_          : { A B : Set } → Par A → (A → Par B) → Par B
fail           : { A : Set } →   Par A
catch          : { A B : Set } → Par A → (A → Par B) → Par B → Par B
assert_then_ : { A B : Set } → Bool → Par A → Par A
```

Informally: A value of type Par A represents a piece of decidably partial computation that either successfully produces a value of type A or fails. The first two constructors return and _>>=_ represent the usual monadic operators [5,8], and fail indicates failed computation. We will also need error handling, which is represented by the catch constructor. The behaviour of our catch is slightly different from the usual one: execution of catch mx f my binds the result of mx to f if mx computes successfully, or goes to execution of my if mx fails to compute. Also we will frequently need to ensure that a Bool value is true before continuing with the rest of a program, so the constructor assert_then_ is included for convenience — execution of assert b then p becomes execution of p when b is true, or fails otherwise.

Formally, the meaning of the constructors of Par is given by the interpreter $runPar : \{A : \mathsf{Set}\} \to \mathsf{Par}\ A \to \mathsf{Maybe}\ A$ shown in Figure 1. We say that $mx : \mathsf{Par}\ A$ computes to $x : A$ exactly when $runPar\ mx \equiv \mathsf{just}\ x$, and that mx fails to compute exactly when $runPar\ mx \equiv \mathsf{nothing}$. The first equality is what we aim to reify as data next.

Our reification of decidably partial computation as the datatype Par and Agda's support of full dependent types allow us to compute for any $mx : \mathsf{Par}\ A$ and $x : A$ a type $mx \mapsto x$ whose inhabitants explain how mx computes to x step by step. Below is a sample of the definition of _↦_:

```
_↦_ : { A : Set } → Par A → A → Set₁
return x    ↦ x′ = x ≡ x′
mx >>= f ↦ y  = (x : _) × (mx ↦ x) × (f x ↦ y)
...
```

That is, return x computes to x' when x is exactly x', and $mx >\!\!>\!\!= f$ computes to y when there exists $x : A$ such that mx computes to x and $f\ x$ computes to y

(and so on). Here our use of the term "computes to" is justified, since we can prove that

$$mx \mapsto x \qquad \text{and} \qquad runPar\ mx \equiv \mathsf{just}\ x$$

are equivalent. That is, a term of type $mx \mapsto x$ can be constructed if and only if $runPar\ mx \equiv \mathsf{just}\ x$, meaning that our reification of $runPar\ mx \equiv \mathsf{just}\ x$ as $mx \mapsto x$ is accurate. We are thus entitled to use $\_\mapsto\_$ for stating various properties involving successful decidably partial computation, in particular well-behavedness of bidirectional transformations in the next section.

3 Bidirectional transformations formalised

We follow the classic (well-behaved) lens approach [2] to bidirectional transformations, but adopt the variant used by Pacheco et al [6]:

Definition 1 (lenses). *A (well-behaved)* lens *between a source type* $S : \mathsf{Set}$ *and a view type* $V : \mathsf{Set}$ *is a pair of functions*

$$put : S \to V \to \mathsf{Par}\ S \qquad \text{and} \qquad get : S \to \mathsf{Par}\ V$$

satisfying the PutGet *law:*

$$put\ s\ v \mapsto s' \quad \text{implies} \quad get\ s' \mapsto v \qquad \text{for all } s, s' : S \text{ and } v : V$$

and the GetPut *law:*

$$get\ s \mapsto v \quad \text{implies} \quad put\ s\ v \mapsto s \qquad \text{for all } s : S \text{ and } v : V.$$

In Agda *the above becomes just a record definition:*

```
record Lens (S V : Set) : Set₁ where
  field
    put : S → V → Par S
    get : S → Par V
    PutGet : {s s' : S} {v : V} → (put s v ↦ s') → (get s' ↦ v)
    GetPut : {s : S} {v : V} →    (get s ↦ v) → (put s v ↦ s)
```

Informally: Execution of $put\ s\ v$ should produce an updated version of the source s by properly embedding all information of the view v into s. The *PutGet* law ensures that *put* does its job properly by requiring that, after *put* updates a source with a view, *get* can completely recover the view from the updated source. The *GetPut* law, on the other hand, ensures that *put* does nothing more than its designated job by requiring that *put* performs no update on a source if the view it receives is directly extracted from the source by *get*. Note that our definition of lenses is stronger than the one given by Foster et al [2]: our *PutGet* law says that successful computation of *put* guarantees success of the

subsequent computation of *get*, whereas in Foster et al's definition there is no such guarantee; the *GetPut* law is similar.

A related notion is "partial" isomorphisms as defined by

```
record Iso (A B : Set) : Set₁ where
  field
    to     : A → Par B
    from   : B → Par A
    to-from-inverse : {x : A} {y : B} → (to x ↦ y) → (from y ↦ x)
    from-to-inverse : {x : A} {y : B} → (from y ↦ x) → (to x ↦ y)
```

Lens is a generalisation of Iso because we can easily convert Iso A B to Lens A B, whose *put* directly produces a new source from its view argument, ignoring its source argument:

```
iso-lens : {A B : Set} → Iso A B → Lens A B
iso-lens iso = record
  { put = λ _ b → Iso.from iso b
  ; get = Iso.to iso
  ; PutGet = Iso.from-to-inverse iso
  ; GetPut = Iso.to-from-inverse iso}
```

This conversion gives us a first way to manufacture lenses. For example, the identity isomorphism between the same type — whose *to* and *from* are both return — gives rise to the "replacing" lens, and the empty isomorphism between any two types — whose *to* and *from* are both $\lambda\ \_ \to$ fail — gives rise to the "failure" lens. Both lenses will appear in Section 4.1.

To give the reader a taste of how well-behavedness proofs are constructed in our Agda setting, let us try to define an operator $\_\leftrightarrow\_$ for lens composition:

```
_↔_ : {A B C : Set} → Lens A B → Lens B C → Lens A C
l ↔ r = record
  { put = λ a c → Lens.get l a >>= λ b → Lens.put r b c >>= Lens.put l a
  ; get = λ a → Lens.get l a >>= Lens.get r
  ; PutGet = ?
  ; GetPut = ?}
```

The *put* and *get* functions for a composite lens can be pretty straightforwardly constructed from the *put* and *get* functions of the component lenses. As part of the definition of $\_\leftrightarrow\_$, we also need to prove that *PutGet* and *GetPut* hold for the pair of *put* and *get* we provide. For *PutGet* we need to show that, for all a, $a' : A$, and $c : C$, from a proof of $put\ a\ c \mapsto a'$ we can construct a proof of $get\ a' \mapsto c$. That is, we need to write a function converting an inhabitant of the type $put\ a\ c \mapsto a'$ to one of the type $get\ a' \mapsto c$. Agda automatically expands the two types following the definition of $\_\mapsto\_$:

```
  put a c ↦ a'
= (b : B) × (Lens.get l a ↦ b) ×
            (b' : B) × (Lens.put r b c ↦ b') × (Lens.put l a b' ↦ a')
```

and

$$\begin{aligned} & \mathit{get}\ a' \ \mapsto\ c \\ = \ & (b : B) \times (\mathsf{Lens}.\mathit{get}\ l\ a' \ \mapsto\ b) \times (\mathsf{Lens}.\mathit{get}\ r\ b \ \mapsto\ c) \end{aligned}$$

Thus all we need to do is convert a five-tuple to a three-tuple of the above types. This is easy: Applying $\mathsf{Lens}.\mathit{PutGet}\ l$ to the fifth component of type $\mathsf{Lens}.\mathit{put}\ l\ a\ b' \ \mapsto\ a'$ we obtain an inhabitant of type $\mathsf{Lens}.\mathit{get}\ l\ a'\ b'$, and applying $\mathsf{Lens}.\mathit{PutGet}\ r$ to the fourth component of type $\mathsf{Lens}.\mathit{put}\ r\ b\ c \ \mapsto\ b'$ we obtain an inhabitant of type $\mathsf{Lens}.\mathit{get}\ r\ b'\ c$. Hence the proof term for *PutGet* is simply

$$\lambda\ (b\ ,\ x\ ,\ b'\ ,\ y\ ,\ z) \to (b'\ ,\ \mathsf{Lens}.\mathit{PutGet}\ l\ z\ ,\ \mathsf{Lens}.\mathit{PutGet}\ r\ y)$$

With a similar reasoning, we can also construct a proof term for *GetPut*:

$$\lambda\ (b\ ,\ x\ ,\ y) \to (b\ ,\ x\ ,\ b\ ,\ \mathsf{Lens}.\mathit{GetPut}\ r\ y\ ,\ \mathsf{Lens}.\mathit{GetPut}\ l\ x)$$

completing the definition of $\_\leftrightarrow\_$. For the proofs in the rest of this paper we will merely provide a sketch, but these proofs all have their formal counterparts in Agda as illustrated above, completely guaranteeing their validity.

4 BiGUL and its lens semantics

Various bidirectional programming languages have been built based on the idea of lenses. Such languages would contain a set of primitive lenses, like those we build by *iso-lens*, and a set of combinators which combine smaller lenses into larger ones, like what $\_\leftrightarrow\_$ does. Usually these languages are designed to suggest that what they describe are *get* programs, but it has recently been argued that the opposite *putback-based* approach should be adopted instead. The argument is based on the following theorem:

Theorem 1 (putback is the essence). *Define, for every* mx, $my : \mathsf{Par}\ A$, *a type* $mx \approx my$ *to mean that* $mx \ \mapsto\ z$ *if and only if* $my \ \mapsto\ z$ *for any* $z : A$. *Then for any* S, $V : \mathsf{Set}$ *and two lenses* l, $r : \mathsf{Lens}\ S\ V$ *such that*

$$\mathsf{Lens}.\mathit{put}\ l\ s\ v \approx \mathsf{Lens}.\mathit{put}\ r\ s\ v \qquad \text{for all } s : S \text{ and } v : V,$$

we have

$$\mathsf{Lens}.\mathit{get}\ l\ s \approx \mathsf{Lens}.\mathit{get}\ r\ s \qquad \text{for all } s : S.$$

The theorem tells us that, when designing a bidirectional combinator, if we can justify the design of its *put* component, then its *get* component needs no further justification, since there is no other *get* that can be paired with the same *put*. This is in sharp contrast with the *get*-based approach, with which the designer needs to choose for each *get* one or a few possible *put* strategies to be paired with the *get*. This complicates the design as there would be multiple combinators with

```
data BiGUL : U → U → Set₁ where
  replace : {S : U} →                 BiGUL S S
  fail    : {S V : U} →               BiGUL S V
  skip    : {S : U} →                 BiGUL S one
  caseS   : {S V : U} →               List (CaseSBranch′ S V) → BiGUL S V
  caseV   : {S V : U} →               List (CaseVBranch′ S V) → BiGUL S V
  align   : {S V : U} →               (source-condition : ⟦ S ⟧ → Par Bool)
                                      (match : ⟦ S ⟧ → ⟦ V ⟧ → Par Bool)
                                      (b : BiGUL S V)
                                      (create : ⟦ V ⟧ → Par ⟦ S ⟧)
                                      (conceal : ⟦ S ⟧ → Par (Maybe ⟦ S ⟧)) → BiGUL (list S) (list V)
  update  : {S : U} →                 (pat : Pattern S) (bs : BiGULs pat) → BiGUL S (Views′ pat bs)
  rearr   : {S V V′ : U} →            (pat : Pattern V) (pat′ : Pattern V′)
                                      (paths : Paths pat pat′) → BiGUL S V′ → BiGUL S V
```

Fig. 2. Simplified definition of BiGUL (auxiliary definitions omitted)

the same *get* semantics, distinguished only by differences in their *put* semantics — this is rather counterintuitive since the programmer is supposed to think in terms of *get* instead of *put* when programming in *get*-based languages. Also it may well happen that, after the programmer describes a *get*, the corresponding *put* synthesised by the language is not what the programmer wants. With the putback-based approach,

. . .

Following the putback-based approach, our language BiGUL describes *put* functions, i.e., updates to a source using a view. In general, an update proceeds by decomposing the source into several parts to be updated independently (Section 4.5), and accordingly rearranging the view to match with these parts (Section 4.6), until the source and the view are reduced to the extent that atomic operations can be applied (Section 4.1). When both the source and the view are lists, there is a powerful aligning operation for synchronising the two lists (Section 4.4). And we can do case analyses on either the source (Section 4.2) or the view (Section 4.3) for describing more flexible update strategies.

In the rest of this section we will present the lens semantics of each of the bidirectional operations mentioned above.

simplified deep embedding; omit definitions of interpreter and (most of) auxiliary types as they are straightforward

Design strategy: start from a natural semantics of *put*, and add checks that guarantee existence of *get*

4.1 Replacing, failure, and skipping

The three most basic operations are replace, fail, and skip. The replace operation is applicable when the source and the view are of the same type, and its semantics is discarding the original source and returning the view as the updated source. As mentioned in Section 3, we can in fact derive its lens semantics from the identity isomorphism:

```
replace-lens : {S : Set} → Lens S S
replace-lens = iso-lens (record {to = return ; from = return ; ...})
```

The *get* semantics for replace is therefore just the identity function. Also mentioned in Section 3 is that we can derive the lens semantics for fail, which fails to compute for any input, from the empty isomorphism in the same way:

```
fail-lens : {S V : Set} → Lens S V
fail-lens = iso-lens (record {to = fail ; from = fail ; ...})
```

Its *get* semantics also fails for any input. On the other hand, the semantics of the skip operation is to discard the view and return the original source, and its lens semantics is not an instance of *iso-lens*:

```
skip-lens : {S : Set} → Lens S ⊤
skip-lens = record {put = λ s _ → return s ; get = λ _ → return tt ; ...}
```

Note that the view type is specified as $\top$, whose inhabitant can only be tt: had the view type been a more complex type, there would have been no way for *get* to decide what to return. In general, *put* must embed all view information into the updated source so *get* can recover the entire view from the updated source, establishing *PutGet*. $\top$ is a type with no information, and hence Lens.*put skip-lens* can safely ignore its view argument of type $\top$.

4.2 Case analysis on source

The next construct caseS is for doing case analysis on the source. The basic idea of caseS's semantics is to choose from a list of branches the first one that matches the source, and then execute the BiGUL statement of that branch. It is, however, difficult to find a *get* to pair with this *put* semantics: the obvious *get* which selects the first branch that matches its source argument does not work, because it may well happen that the updated source produced by *put* matches a different branch from the original source, and hence in general we would have to guarantee *PutGet* for *put* and *get* coming respectively from any two branches, which is unmanageable. A more manageable solution is to insert a dynamic check at the end of *put* to ensure that the updated source must match the same branch as the original source, which is the solution adopted by Foster et al's concrete conditional lenses [2, Section 6.1]. This solution, however, is too restrictive to be useful in practice.

Our solution is to enhance caseS by adding a special kind of branches called *adaptive* branches, in addition to *normal* branches. The lens semantics of the two kinds of branches are defined by the following datatype:

```
data CaseSBranchType (S V : Set) : Set₁ where
  normal   : Lens S V      → CaseSBranchType S V
  adaptive : (S → Par S) → CaseSBranchType S V
```

```
caseS-lens : (S V : Set) → List (CaseSBranch S V) → Lens S V
caseS-lens S V bs = record
  { put = put-with-adaptation bs s v
            (λ s′ → put-with-adaptation bs s′ v (λ _ → fail))
  ; get = get ; ... }
  where
    branch : {X : Set₁} → (Lens S V → X) → ((S → Par S) → X) →
             CaseSBranchType → X
    branch f g (normal   l) = f l
    branch f g (adaptive u) = g u

    put-with-adaptation : List CaseSBranch → S → V → (S → Par S) → Par S
    put-with-adaptation []             s v f = fail
    put-with-adaptation ((p , b) :: bs) s v f =
      p s >>= λ matched →
      if matched then branch (λ lens → Lens.put lens s v >>= λ s′ →
                                       p s′ >>= λ matched′ →
                                       assert matched′ then return s′)
                             (λ u → u s >>= f) b
                 else put-with-adaptation bs s v f >>= λ s′ →
                      p s′ >>= λ matched′ → assert not matched′ then return s′

    get : List CaseSBranch → S → Par V
    get []             s = fail
    get ((p , b) :: bs) s =
      p s >>= λ matched →
      if matched then branch (λ lens → Lens.get lens s) (λ _ → fail) b
                 else get bs s
```

Fig. 3. Definition of source case analysis

The lens semantics of a normal branch is just a lens, while for an adaptive branch it is a source transformation. With each branch we also need to associate a decidable predicate on the source type specifying when a source matches the branch, so the complete definition of the type of branches is

```
CaseSBranch : Set → Set → Set₁
CaseSBranch S V = (S → Par Bool) × CaseSBranchType S V
```

The type of caseS's lens semantics *caseS-lens* is then

```
caseS-lens : (S V : Set) → List (CaseSBranch S V) → Lens S V
```

and its definition is shown in Figure 3. We might think of a source case analysis as still consisting mainly of normal branches, but we can now specify additional cases such that when a source matches none of the normal branches, the source can still be transformed, or adapted, to one that matches a normal branch. More explicitly, the case analysis may be run twice: if a normal branch is chosen the first time, then the case analysis succeeds and we execute the branch; otherwise,

```
caseV-lens : (S V : Set) → DecEq V → List CaseVBranch S V → Lens S V
caseV-lens S V dec bs = record {put = put ; get = get ; ... }
  where
    guarded-return : S → V → V → Par S
    guarded-return s' v v' with dec v v'
    guarded-return s' v v' | inj₁ eq  = return s'
    guarded-return s' v v' | inj₂ neq = fail

    put : List CaseVBranch S V → S → V → Par S
    put []                       s v = fail
    put ((_ , lens , iso) :: bs) s v = catch (Iso.from iso v) (Lens.put lens s)
                                             (put bs s v >>= λ s' →
                                              catch (Lens.get lens s' >>= Iso.to iso)
                                                    (guarded-return s' v) (return s'))

    get : List CaseVBranch S V → S → Par V
    get []                       s = fail
    get ((_ , lens , iso) :: bs) s = catch (Lens.get lens s >>= Iso.to iso) return
                                           (get bs s >>= λ v →
                                            catch (Iso.from iso v) (λ _ → fail) (return v))
```

Fig. 4. Definition of view case analysis

we adapt the source and rerun the case analysis, and must match a normal branch this time. After a normal branch is executed, we must ensure that the updated source satisfies the predicate of the branch and also none of the predicates of the previous branches, so a subsequent execution of *get* on the updated source would choose the same branch as *put*, establishing *PutGet*.

4.3 Case analysis on view

For caseV, basically we still match the view with a list of branches, and execute the first branch matched. We must be more careful when it comes to the view, though: As we mentioned in Section 4.1, view information must be carefully managed and completely embedded into the source. When the view matches a branch, the amount of view information that remains to be embedded into the source can actually be reduced. For example, if there is a branch that matches an integer view which equals zero, then for this branch we may even perform no update on the source as long as we can guarantee that the subsequent *get* will choose this branch, in which case *get* can simply return zero — the information that the view is zero is embedded implicitly in the control flow rather than explicitly in the updated source. Thus, in the definition of CaseVBranch below, we use a partial isomorphism on the view type which converts it to a less informative type, instead of just a decidable predicate as in CaseSBranch:

```
CaseVBranch : Set → Set → Set₁
CaseVBranch S V = (V' : Set) × Lens S V' × Iso V' V
```

For the comparison-with-zero example above, we would set V' to be $\top$, indicating that there is no more information to be embedded into the view after a successful matching, and set the *to* and *from* components of the isomorphism to $\lambda \_ \to \mathsf{return}\ 0$ and $\lambda\ n \to \mathsf{assert}\ n == 0\ \mathsf{then}\ \mathsf{return}\ \mathsf{tt}$. A branch is considered matched with a view when the *from* conversion of the associated isomorphism succeeds on the view.

We can now define the lens semantics of caseV:

$$\mathit{caseV\text{-}lens} : (S\ V : \mathsf{Set}) \to \mathsf{DecEq}\ V \to \mathsf{List}\ \mathsf{CaseVBranch} \to \mathsf{Lens}\ S\ V$$

and its definition is shown in Figure 4. (We will come to the DecEq requirement shortly.) The main behaviour of its *put* is to try to match the view with each branch and execute the first successful branch. For *get*, however, there is no obvious way to decide which branch to go, as we are not given a view to start with like *put*. It seems that we can only try to execute each of the branches until we reach one that computes a view successfully, which is the approach taken by Pacheco et al [6]. This makes guaranteeing *PutGet* very difficult, as we cannot easily guarantee that executing *put* followed by *get* would go through the same branch. Our compromised solution is to make *put* do an expensive check after a branch is matched and executed, requiring that the *get* for each of the previous branches must either fail or produce the same view as the input one (i.e., verifying *PutGet* directly). That is, the ranges of the branches have to be "more or less" disjoint in order for *put* to compute successfully. The DecEq V argument of *caseV-lens*, which expands to $(v\ v' : V) \to (v \equiv v') \uplus \neg\ (v \equiv v')$ and says that V is equipped with a decidable equality, is needed for this check.

4.4 List alignment

4.5 Source update

A fundamental operation is performing updates to parts of the source. We follow the approach taken by Flux [1], which requires that updates must be independent — that is, the same part of a source cannot be updated more than once — to avoid the problem of sequencing updates. It turns out that a pattern matching notation suits this task perfectly: pattern matching is arguably the most intuitive way to deconstruct a source, and we can specify apparently independent updates by writing them at the variable positions in a pattern. The idea itself is straightforward; what follows, despite its slight complexity (especially if the reader is not familiar with dependently typed programming), is merely our strongly typed implementation of the idea in Agda.

Since Agda is fully dependently typed, we can naturally define patterns in a type-directed way such that nonsensical patterns are syntactically ruled out. This task starts from the construction of a *universe*, i.e., a datatype whose inhabitants are interpreted as types. The actual Agda implementation of BiGUL uses a universe capable of expressing mutually inductive datatypes, but, to make the presentation easier to follow, below we present only a simplified universe U, along with its interpreter $[\![\_]\!]$:

```
data U : Set₁ where
  k     : (A : Set) → DecEq A → U
  one   : U
  _⊕_   : U → U → U
  _⊗_   : U → U → U
  list  : U → U → U

⟦_⟧ : U → Set
⟦ k A _  ⟧ = A
⟦ one    ⟧ = ⊤
⟦ F ⊕ G  ⟧ = ⟦ F ⟧ ⊎ ⟦ G ⟧
⟦ F ⊗ G  ⟧ = ⟦ F ⟧ × ⟦ G ⟧
⟦ list F ⟧ = List ⟦ F ⟧
```

The types that we can encode within U are those expressible in terms of any types with a decidable equality, $\top$, $\_\uplus\_$, $\_\times\_$, and List; applying $[\![\_]\!]$ to an inhabitant of the universe decodes it to the type it represents. All encoded types are equipped with a decidable equality, as we can define a function of type

```
U-dec : (D : U) → DecEq ⟦ D ⟧
```

by induction on D. We can now define a family of types Pattern : U → Set such that, for any D : U, the type Pattern D contains exactly those patterns sensible for $[\![D]\!]$:

```
data Pattern : U → Set where
  var   : {D : U} →          Pattern D
  const : {D : U} →          ⟦ D ⟧ → Pattern D
  left  : {D E : U} →        Pattern D → Pattern (D ⊕ E)
  right : {D E : U} →        Pattern E → Pattern (D ⊕ E)
  prod  : {D E : U} →        Pattern D → Pattern E → Pattern (D ⊗ E)
  cons  : {D : U} →          Pattern D → Pattern (list D) → Pattern (list D)
```

On all types we can use var patterns, which match anything, and const patterns, which match inhabitants equal to the specified constant (this is possible because of *U-dec*); on sum types we can use left or right patterns matching inj_1 and inj_2 constructors; on product types we can use prod patterns to decompose a pair to its two components; on list types we can use cons patterns to decompose a non-empty list to its head and tail.

Next we will interpret a pattern as a "container" and store values — whose types depend on the types of the variable patterns — at its variable positions. The types of such containers can be defined by induction on Pattern:

```
VarPositions : {lv : Level} {D : U} → Pattern D → (U → Set lv) → Set lv
VarPositions {D} var    = f D
VarPositions (const _)  = ⊤
VarPositions (left  pat) = VarPositions pat
```

$$\begin{aligned}
&\mathsf{VarPositions}\ (\mathsf{right}\ pat) = \mathsf{VarPositions}\ pat\\
&\mathsf{VarPositions}\ (\mathsf{prod}\ lpat\ \ rpat) = \mathsf{VarPositions}\ lpat\ \times \mathsf{VarPositions}\ rpat\\
&\mathsf{VarPositions}\ (\mathsf{cons}\ hpat\ tpat) = \mathsf{VarPositions}\ hpat \times \mathsf{VarPositions}\ tpat
\end{aligned}$$

(In fact, all entities below that depend on Pattern are defined by a similar induction.) For example, the result of a successful pattern matching can be filled into such a container, by putting the resulting components at their respective variable positions. We thus define the types of pattern matching results as an instance of VarPositions:

$$\begin{aligned}
&\mathsf{PatResult} : \{\, D : \mathsf{U} \,\} \to \mathsf{Pattern}\ D \to \mathsf{U}\\
&\mathsf{PatResult}\ pat\ = \mathsf{VarPositions}\ pat\ [\![_]\!]
\end{aligned}$$

Pattern matching is then simply a partial isomorphism:

$$pat\text{-}iso : \{\, D : \mathsf{U} \,\} \to \mathsf{Pattern}\ D \to \mathsf{Iso}\ [\![\ D\ ]\!]\ (\mathsf{PatResult}\ pat)$$

The *to* component of the isomorphism performs pattern matching (which can fail because matching with left and right patterns may not succeed), and the *from* component reverses pattern matching by evaluating a pattern as if it is an expression (which always succeeds), using the input of type PatResult *pat* as an environment for values at variable positions.

Similarly, by specialising VarPositions we can place lenses (and their view types) at the variable positions of a pattern:

$$\begin{aligned}
&\mathsf{Lenses} : \{\, D : \mathsf{U} \,\} \to \mathsf{Pattern}\ D \to \mathsf{Set}_1\\
&\mathsf{Lenses}\ pat\ = \mathsf{VarPositions}\ pat\ (\lambda\ D \to (E : \mathsf{U}) \times \mathsf{Lens}\ [\![\ D\ ]\!]\ [\![\ E\ ]\!])
\end{aligned}$$

Given a pattern *pat* and a collection of lenses ls : Lenses *pat* at its variable positions, we can define a lens between PatResult *pat* and a type Views *pat* ls that combines all the view types used by the lenses:

$$\begin{aligned}
&pat\text{-}lens : \{\, D : \mathsf{U} \,\}\ (pat : \mathsf{Pattern}\ D)\ (ls : \mathsf{Lenses}\ pat) \to\\
&\qquad \mathsf{Lens}\ (\mathsf{PatResult}\ pat)\ (\mathsf{Views}\ pat\ ls)
\end{aligned}$$

where the definition of Views is as straightforward as

$$\begin{aligned}
&\mathsf{Views} : \{\, D : \mathsf{U} \,\}\ (pat : \mathsf{Pattern}\ D) \to \mathsf{Lenses}\ pat \to \mathsf{Set}\\
&\mathsf{Views}\ \{\, D \,\}\ \mathsf{var}\quad (E\ ,\ _) = [\![\ E\ ]\!]\\
&\mathsf{Views}\ (\mathsf{const}\ _)\quad ls\ = \top\\
&\mathsf{Views}\ (\mathsf{left}\quad pat)\ ls\ = \mathsf{Views}\ pat\ ls\\
&\mathsf{Views}\ (\mathsf{right}\ pat)\ ls\ = \mathsf{Views}\ pat\ ls\\
&\mathsf{Views}\ (\mathsf{prod}\ lpat\ \ rpat)\ (ls\ ,\ ls') = \mathsf{Views}\ lpat\ \ ls \times \mathsf{Views}\ rpat\ ls'\\
&\mathsf{Views}\ (\mathsf{cons}\ hpat\ tpat)\ (ls\ ,\ ls') = \mathsf{Views}\ hpat\ ls \times \mathsf{Views}\ tpat\ ls'
\end{aligned}$$

The lens semantics of the update operation is then just the composition of *pat-iso* and *update-lens*. Its *put* decomposes the source, updates its components with the lenses at the corresponding variable positions, and putting the updated components together, while its *get* again decomposes the source and extracts views from its components with the lenses.

4.6 View rearrangement

The last operation rearr is for rearranging the view to a specific shape suitable for subsequent updates. For example, the update operation in Section 4.5 demands that the view must be in a shape that matches the source pattern, and this is usually achieved by an outer rearr. In essence, rearr performs a piece of invertible computation (which loses no information), while its *get* semantics performs the inverse of the computation. The invertible computation used is usually not complicated (hence the name "rearrangement"), so instead of letting the programmer write general invertible functions (perhaps along with their inverses), we ask instead for syntactic descriptions of simple invertible computation, which are more restrictive but allow automatic inversion.

The kind of invertible computation we wish to express is a decomposition of an old view followed by an assembling of a new view using all components of the old view. When it comes to decomposition, pattern matching should come to mind again; assembling can also be handled by pattern matching, as we have formulated pattern matching as an isomorphism in Section 4.5. We thus require two patterns pat : Pattern D and pat' : Pattern E, the former for decomposing the old view of type $[\![\ D\ ]\!]$ and the latter for assembling the new view of type $[\![\ E\ ]\!]$. For the latter pattern pat' we also need to specify which component of the old view will be used as the value at each variable position. This last piece of information is represented by specialising VarPositions again:

$$\begin{aligned}&\mathsf{Paths} : \{\, D\ E : \mathsf{U} \,\} \to \mathsf{Pattern}\ D \to \mathsf{Pattern}\ E \to \mathsf{Set}_1\\ &\mathsf{Paths}\ pat\ pat' = \mathsf{VarPositions}\ pat'\ (\mathsf{Path}\ pat)\end{aligned}$$

where Path pat T is a path that leads to a component of type $[\![\ T\ ]\!]$ at a variable position of a PatResult pat:

$$\begin{aligned}&\mathsf{Path} : \{\, D : \mathsf{U} \,\} \to \mathsf{Pattern}\ D \to \mathsf{U} \to \mathsf{Set}_1\\ &\mathsf{Path}\ \{\, D \,\}\ \mathsf{var} && T = D \equiv T\\ &\mathsf{Path}\ (\mathsf{k}\ \_) && T = \bot\\ &\mathsf{Path}\ (\mathsf{left}\ \ pat) && T = \mathsf{Path}\ pat\ T\\ &\mathsf{Path}\ (\mathsf{right}\ pat) && T = \mathsf{Path}\ pat\ T\\ &\mathsf{Path}\ (\mathsf{prod}\ lpat\ \ rpat) && T = \mathsf{Path}\ lpat\ \ T \uplus \mathsf{Path}\ rpat\ T\\ &\mathsf{Path}\ (\mathsf{cons}\ hpat\ tpat) && T = \mathsf{Path}\ hpat\ T \uplus \mathsf{Path}\ tpat\ \ T\end{aligned}$$

If the $paths$: Paths pat pat' supplied uses up all of the old view, i.e., all possible paths for pat are contained in $paths$, then pat, pat', and $paths$ together specify a piece of computation that can be automatically inverted — since all components of the old view are present somewhere in the new view, we can always reassemble the old view from the new view (unless there is inconsistency — a component of the old view might be copied into more than one positions of the new view, and the values at these positions of the new view must be the same when doing the reassembling). In short, we can construct an isomorphism:

$$\begin{aligned}&\mathit{view\text{-}rearrangement\text{-}iso} :\\ &\quad \{\, D : \mathsf{U} \,\}\ (pat : \mathsf{Pattern}\ D)\ \{\, E : \mathsf{U} \,\}\ (pat' : \mathsf{Pattern}\ E) \to\\ &\quad (paths : \mathsf{Paths}\ pat\ pat') \to \mathsf{Invertible}\ pat\ pat'\ paths \to \mathsf{Iso}\ [\![\ E\ ]\!]\ [\![\ D\ ]\!]\end{aligned}$$

where Invertible *pat pat′ paths* is defined to state that *paths* uses up all of the old view. The lens derived from this isomorphism is the semantics we assign to rearr.

5 Examples

bookstore for align, update, and rearrangement

BiYacc for source and view cases (probably not)

source/view if, map in terms of align

6 Discussion

recursion (the lack thereof)

extensions: for example, iteration, view dependency

design/programming phase: design phase needs to consider *put* and *get* together, but programming phase is about *put* only

Haskell implementation

Conditions for *put* to succeed; deferred as we will have a dependently typed version (a completely formal approach to total lenses)

stems from work on BiFluX; comparison with putlenses

Theory construction in terms of Agda programming
Regarding formalisation: "[W]e are interested in extending what can be calculated precisely because we are not interested in the calculations themselves. Developing techniques that allow more of a derivation to be calculated allows attention to be focussed on the creative parts, which cannot be calculated." [3]
[7]

References

1. James Cheney. Flux: functional updates for XML. In *International Conference on Functional Programming*, ICFP'08, pages 3–14. ACM, 2008.
2. J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems*, 29(3):17, 2007.
3. Jeremy Gibbons. An introduction to the Bird-Meertens formalism. In *New Zealand Formal Program Development Colloquium*, pages 1–12, 1994.
4. Jeremy Gibbons. Calculating functional programs. In *Algebraic and Coalgebraic Methods in the Mathematics of Program Construction*, number 2297 in Lecture Notes in Computer Science, chapter 5, pages 149–201. Springer-Verlag, 2002.
5. Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.

6. Hugo Pacheco, Zhenjiang Hu, and Sebastian Fischer. Monadic combinators for "putback" style bidirectional programming. In *Partial Evaluation and Program Manipulation*, PEPM'14, pages 39–50. ACM, 2014.
7. The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, Princeton, 2013.
8. Philip Wadler. The essence of functional programming. In *Principles of Programming Languages*, POPL'92, pages 1–14. ACM, 1992.

Appendix: AGDA syntax

An AGDA datatype is defined by listing all of its constructors along with their types in full. A datatype can be *parametrised*, as in the case of Maybe, which has a parameter $A : \mathsf{Set}$, specified between the name of the datatype and the colon on the first line. (Set is the type of all (small) types.) Parameters can be referred to throughout a datatype definition.

Here the datatype Par is *indexed* by Set (which appears to the right of the colon on the first line), meaning that Par is in fact a Set-indexed family of simultaneously defined types.

As for the constructors, let us look at the simplest case, fail: The constructor fail constructs a value of type Par A for any $A : \mathsf{Set}$. There is thus one argument to fail, namely $A : \mathsf{Set}$, and the type of fail's result depends on the value of this argument (which happens to be a type). This is an example of *dependent function types*, whose result type can depend on the value of the argument. Logically, dependent function types are read as universally quantified propositions — the dependent function type $\{x : A\} \to B\ x$ is read as "for all $x : A$ the property B holds for x". (Ordinary function types, on the other hand, denote implications logically.)

Back to fail: most of the time AGDA can infer what fail's argument should be based on how fail is used, so we make the argument *implicit* by enclosing it in curly brackets. (Explicit arguments, on the other hand, are enclosed in round brackets.) We can then invoke fail without supplying the argument, leaving it for AGDA to infer. We de-emphasise implicit arguments typographically when they cause visual annoyance, as in this case.

An AGDA name can be used as an infix (or, in general, mixfix) operator when the name contains underscores, in which case the underscores indicate where the arguments should go. For example, we can apply $\_\gg\!=\_$ to arguments mx and f by writing either $\_\gg\!=\_\ mx\ f$ or $mx\ \gg\!=\ f$.

The type $\top : \mathsf{Set}$ has exactly one constructor $\mathsf{tt} : \top$, while the type $\bot : \mathsf{Set}$ has no inhabitant — logically they are read as truth and falsity respectively. The type family $\_\equiv\_ : \{A : \mathsf{Set}\} \to A \to A \to \mathsf{Set}$ is defined such that $x \equiv y$ is inhabited if and only if x and y are equal. The type former $\_\uplus\_ : \mathsf{Set} \to \mathsf{Set} \to \mathsf{Set}$ is disjoint union (disjunction) whose constructors are $\mathsf{inj}_1 : {}^{\{A\ B\ :\ \mathsf{Set}\}\ \to}\ A \to A \uplus B$ and $\mathsf{inj}_2 : {}^{\{A\ B\ :\ \mathsf{Set}\}\ \to}\ B \to A \uplus B$, and the type former $\_\times\_ : \mathsf{Set} \to \mathsf{Set} \to \mathsf{Set}$ is cartesian product (conjunction), whose constructor is $\_,\_$. The symbol '$\times$' is also overloaded to denote *dependent pair types* — the inhabitants of a dependent pair type $(x : A) \times B\ x$ are pairs where the type of the second component depends on the value of the first component. Logically, dependent pair types are read as existentially quantified propositions — $(x : A) \times B\ x$ is read as "there exists $x : A$ such that the property B holds for x".

An underscore in a right-hand side expression instructs AGDA to infer what should be placed there.

An AGDA record is defined by listing the names and types of all its components, and a component type can depend on previous components. To access a component of an inhabitant of a record type, we should use the corresponding

component extraction function. For example, $\mathsf{Lens}.get : {}^{\{S\ V\ :\ \mathsf{Set}\}} \rightarrow \mathsf{Lens}\ S\ V \rightarrow S \rightarrow \mathsf{Par}\ V$ is for extracting the *get* component of a Lens.